

Artificial Intelligence

Solving Problem by Searching



Today's Topic

- Informed Search Strategies: one that uses problem specific knowledge
 - Best First Search & its variant
 - Heuristic Functions
 - Local Search and Optimization



Best-first search

- Idea: use an **evaluation function** $f(n)$ for each node
 - ❑ estimate of "desirability"
 - ❑ Expand most desirable unexpanded node
 - ❑ Evaluation function estimates distance to the goal
- Implementation:
Fringe is a priority queue sorted in ascending order of f-values
- Special cases:
 - greedy best-first search
 - A* search

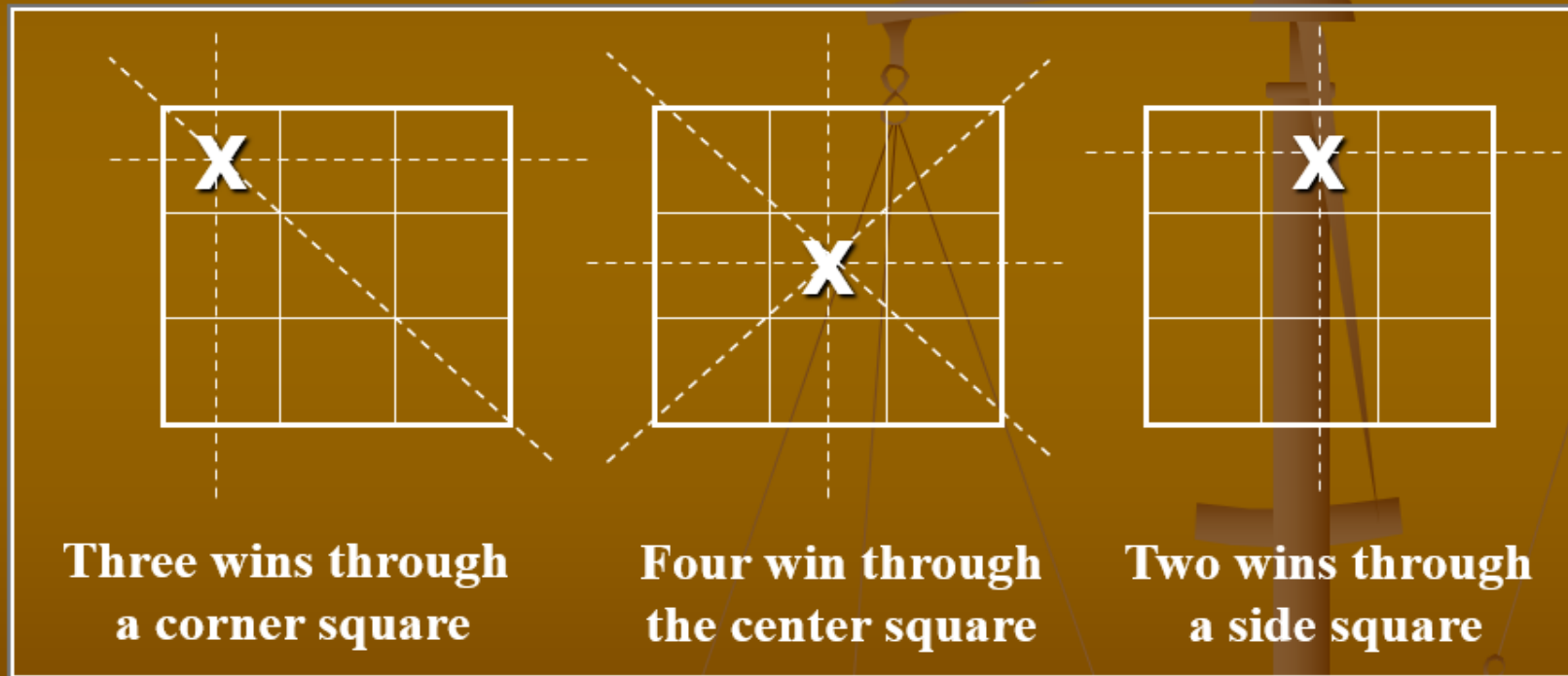


Heuristic Function

- It maps each state to a numerical value depicts goodness of a node.
 - $h(n) = \text{value}$, where $h()$ is heuristic function and n is current state
- It is estimated cost of cheapest path from initial node to goal
- Example: (In Romania)
 - Straight line distance from Arad to Bucharest
 - Heuristic functions are most common way in which additional knowledge of the problem is imparted to search algorithms

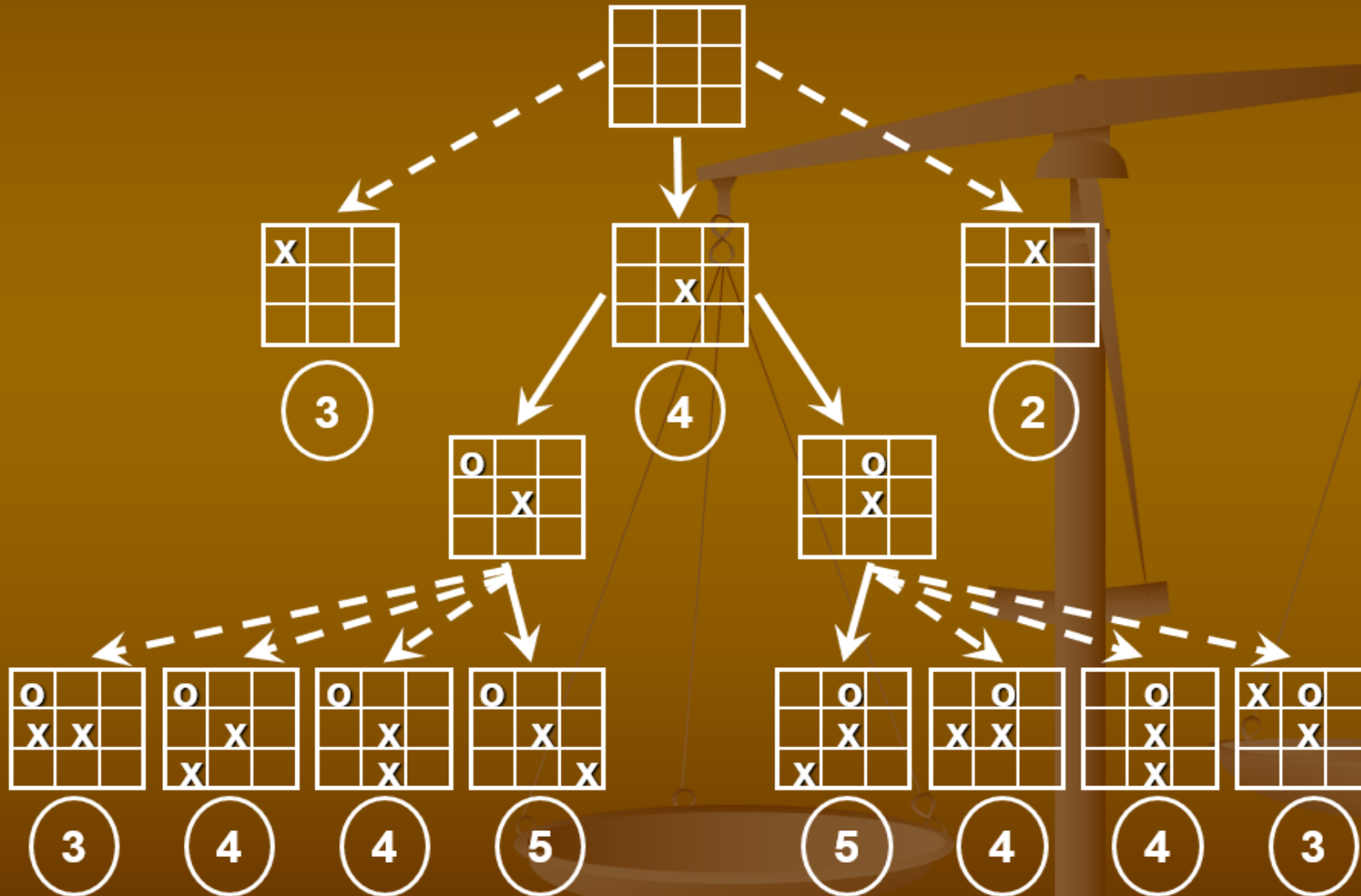
How Heuristic Search Methods Can Be Used?

- A simple heuristic in the game of tic-tac-toe could be:
We may move to the board in which X has the most winning lines

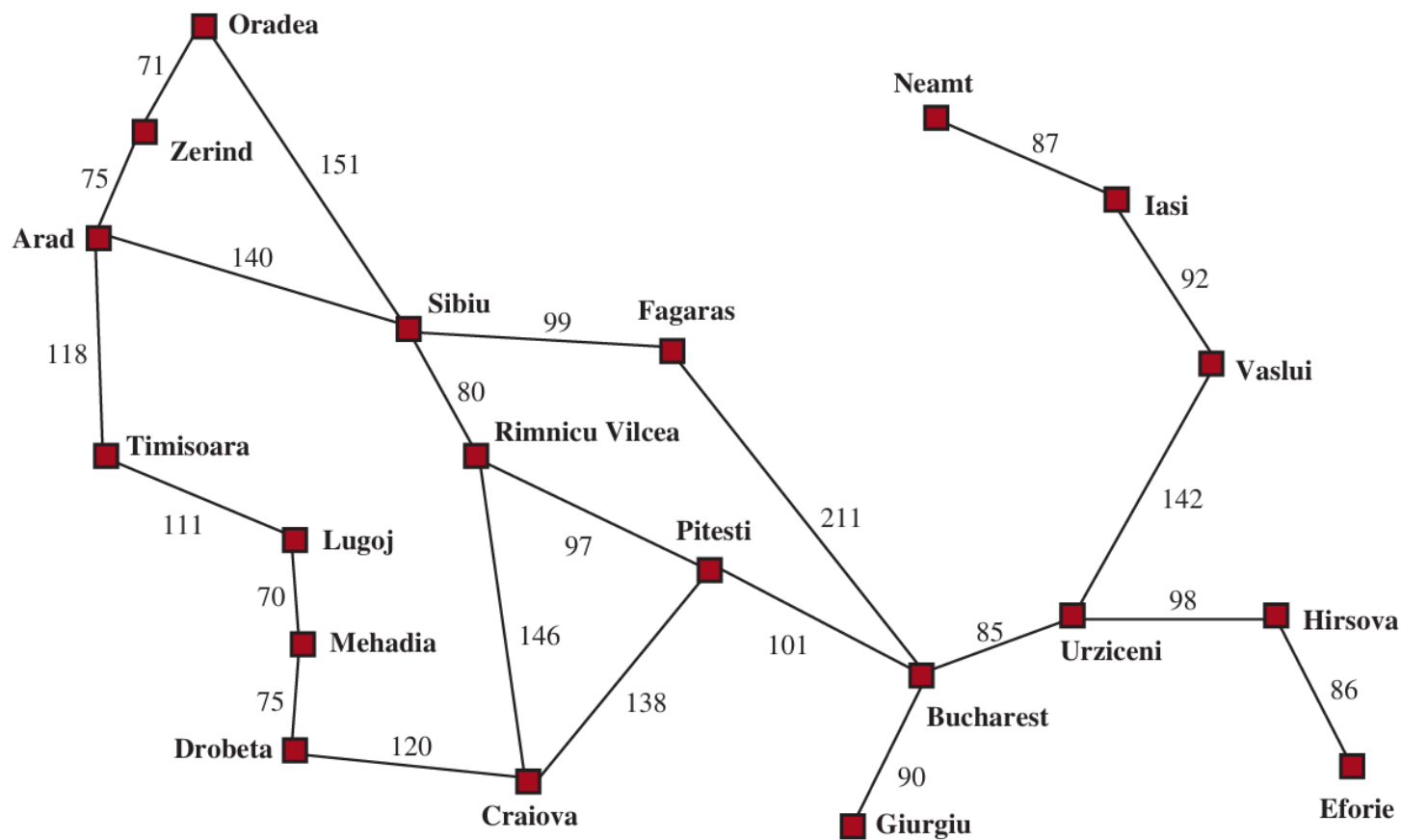


How Heuristic Search Methods Can Be Used?

- Heuristically reduced state space for tic-tac-toe



Romania map in kms



Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Figure 2: Straight line distance

Figure 1: Romania Map with distance in kms



Greedy best-first search

- Evaluation function $f(n) = h(n)$ (**h**euristic)
= estimate of cost from n to *goal*
- e.g., $h_{SLD}(n)$ = straight-line distance from n to Bucharest
- Greedy best-first search expands the node that **appears** to be closest to goal
 - $f(n)=h(n)$

Greedy best-first search

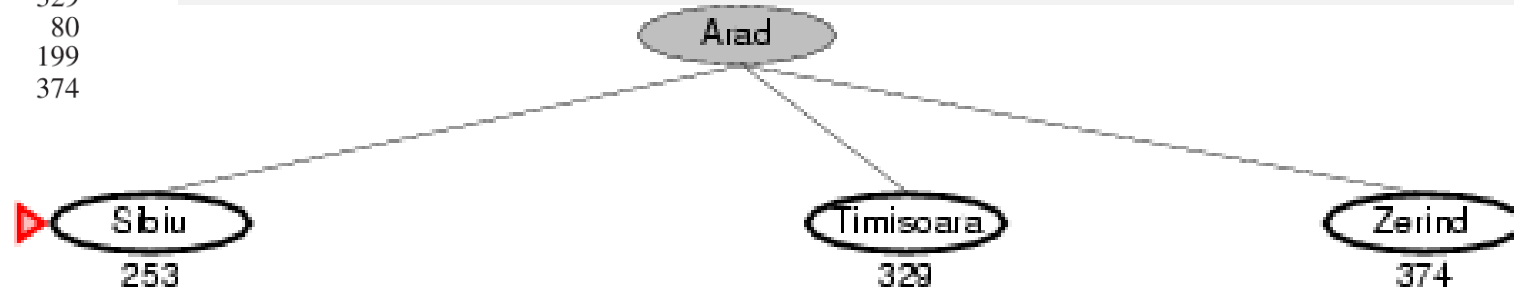
example

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

▶ Arad
366

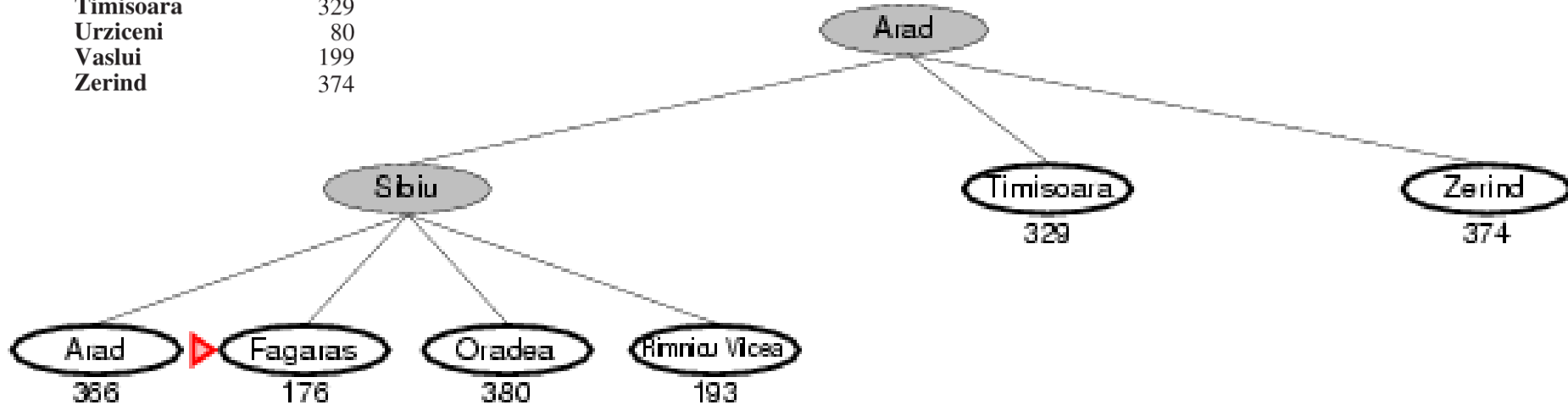
Greedy best-first search example

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



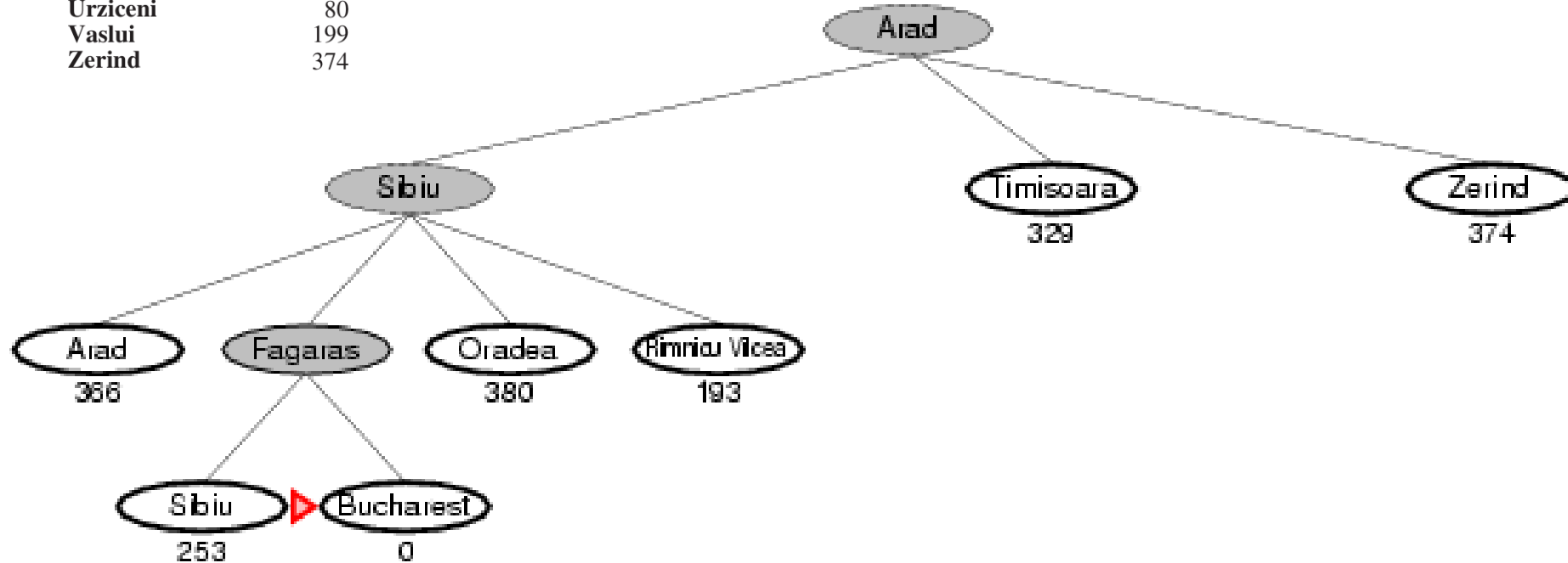
Greedy best-first search example

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

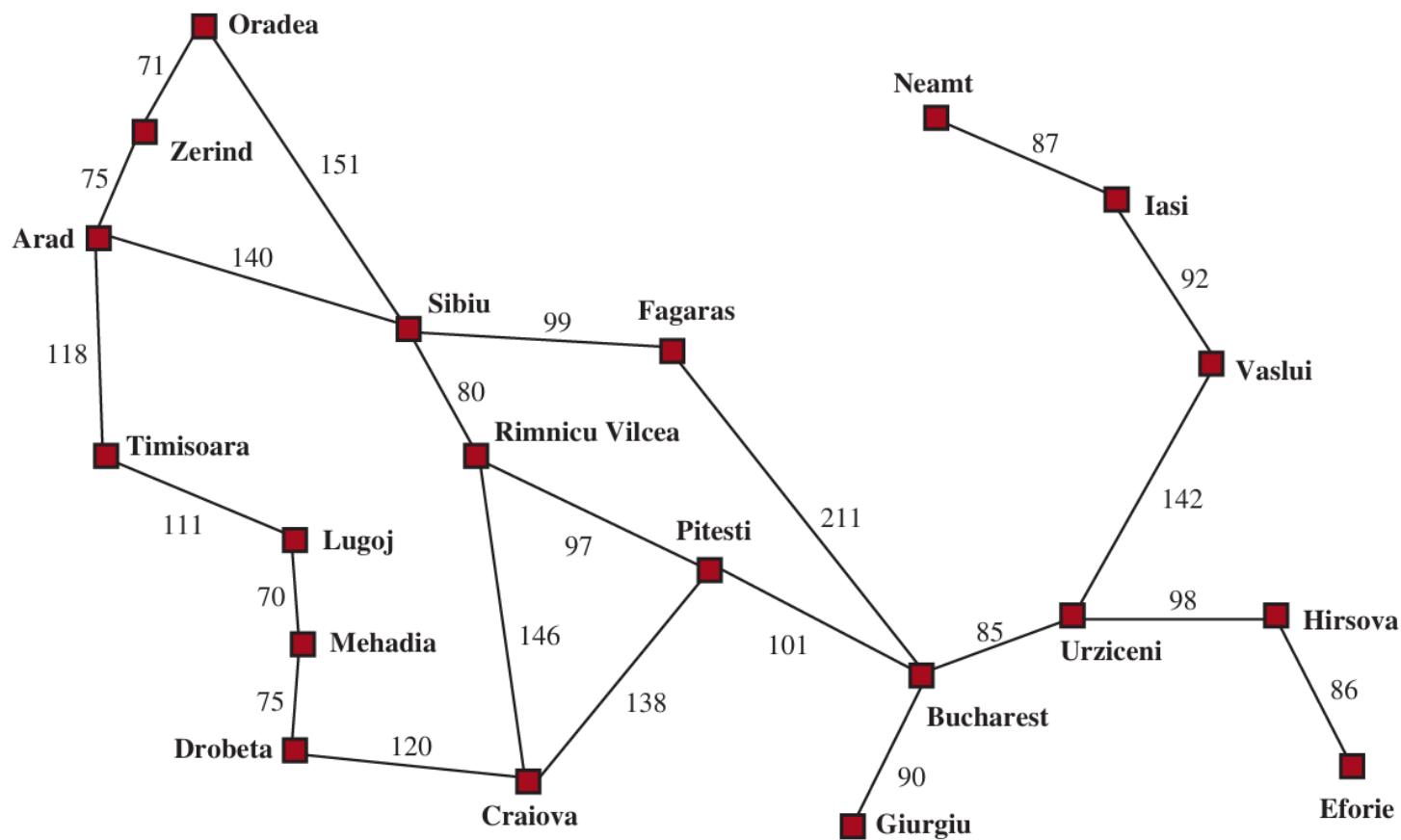


Greedy best-first search example

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



Romania map in kms



Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Figure 2: Straight line distance

Figure 1: Romania Map with distance in kms

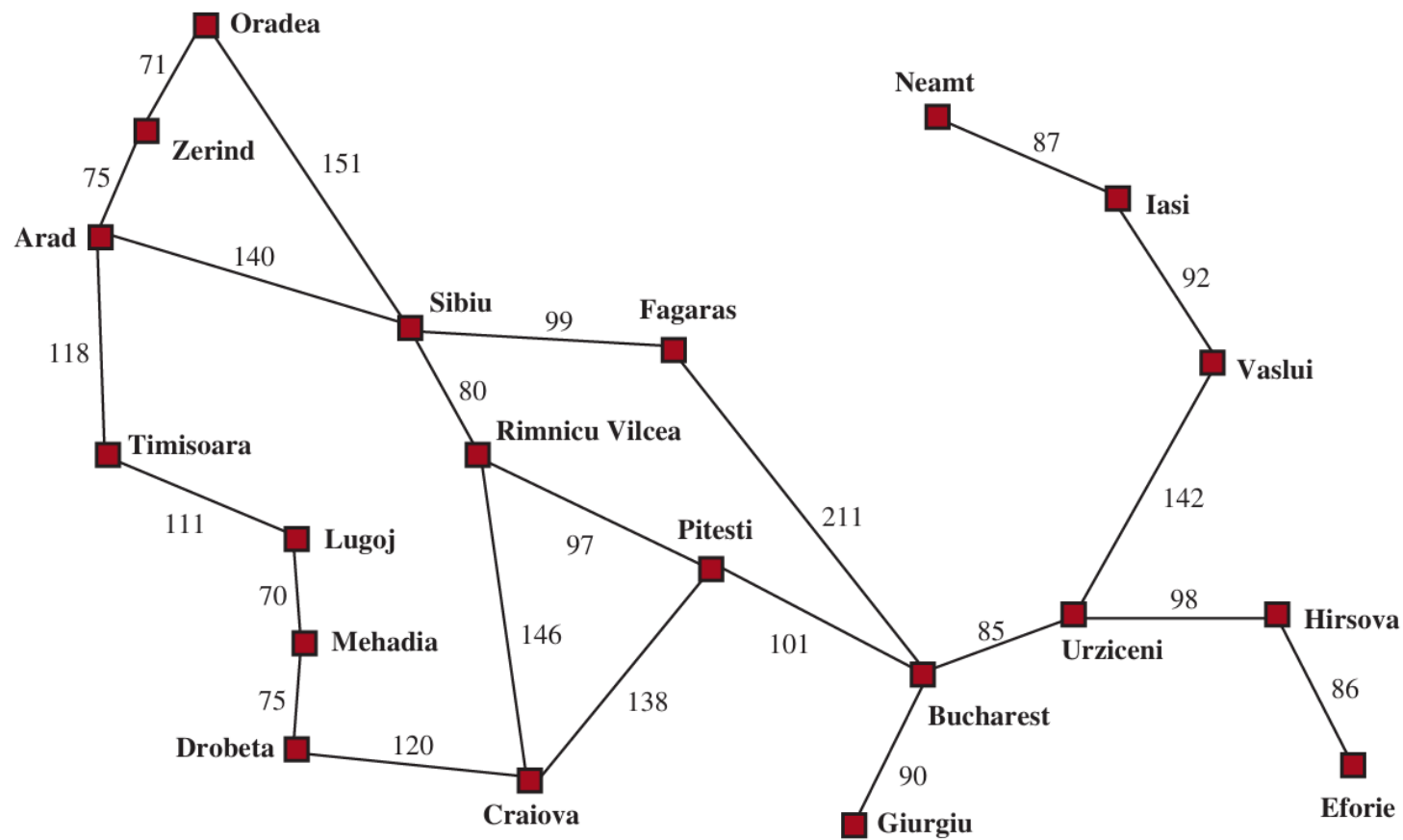


Properties of greedy best-first search

- Complete? No – can get stuck in loops (i.e. Bucharest is unavailable for first solution) No close list.
- Time? $O(b^m)$, but a good heuristic can give dramatic improvement
- Space? $O(b^m)$ -- keeps all nodes in memory
- Optimal? No

Q1: Find path between Arad to Bucharest?

Romania map in kms



Bucharest	0
Arad	360
Sibiu	250
Fagaras	180
Rimnicu Vilcea	200
Oradea	300
Zerind	340
Timisoara	330

Figure 2: Straight line distance

Figure 1: Romania Map with distance in kms



A* search

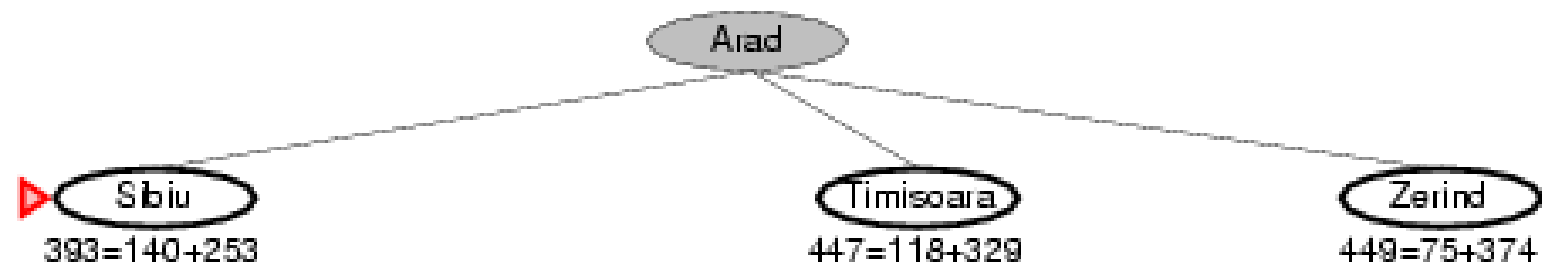
- Idea: avoid expanding paths that are already expensive
- Evaluation function $f(n) = \underline{g(n)} + \underline{h(n)}$
 - $g(n)$ = cost from start/node to n
 - $h(n)$ = estimated cost from n to goal node
 - $f(n)$ = estimated total cost of path through n to goal



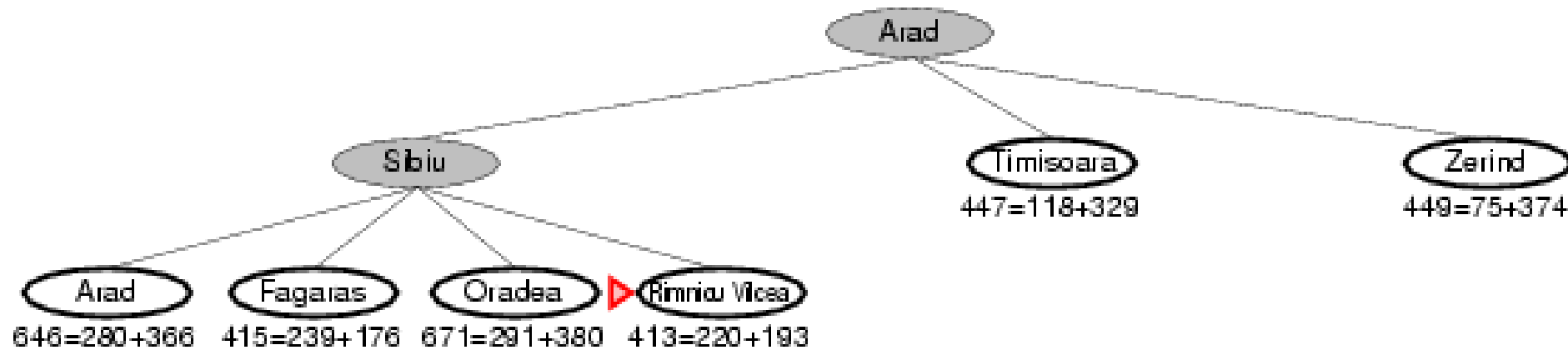
A* search example

▶ Arad
366=0+366

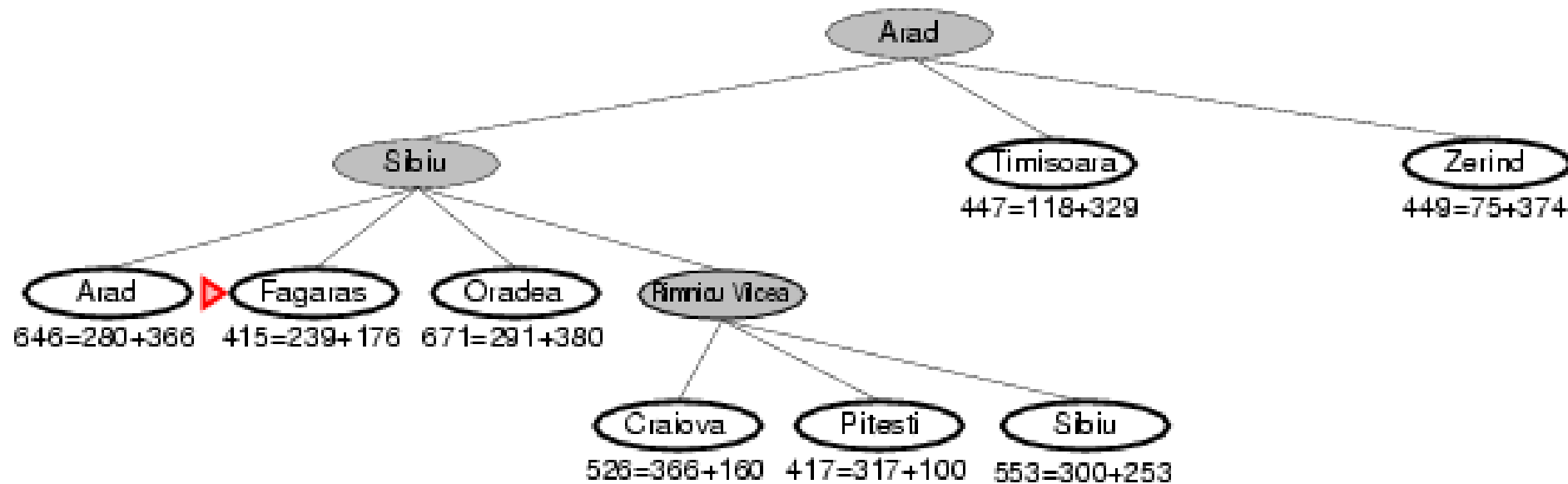
A* search example



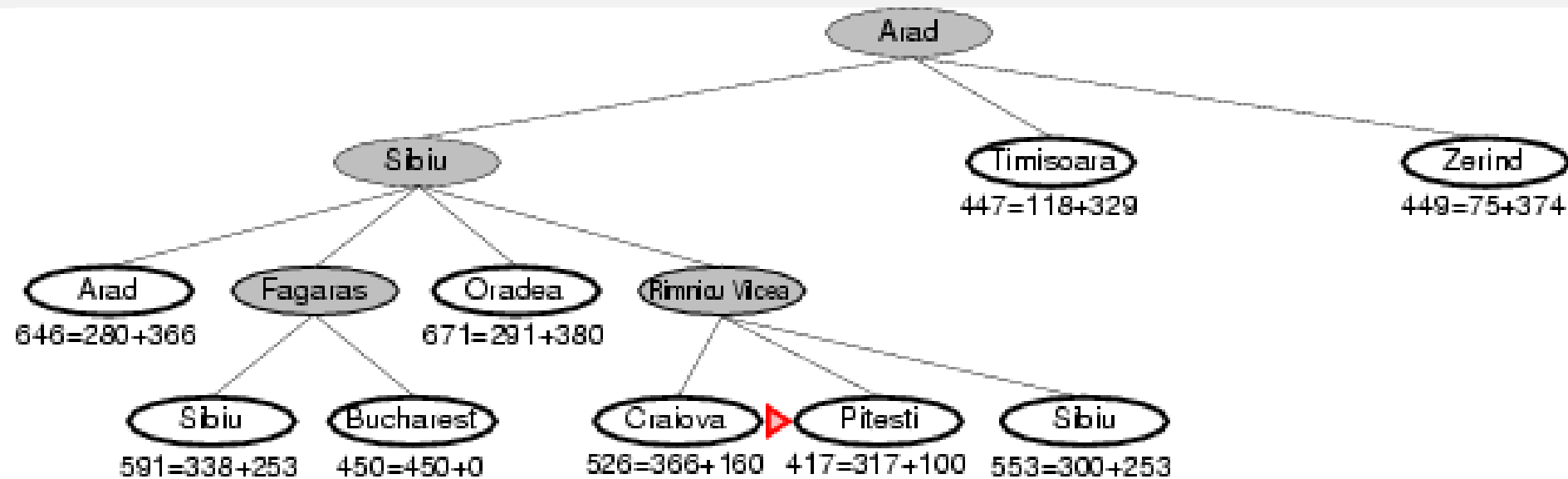
A* search example



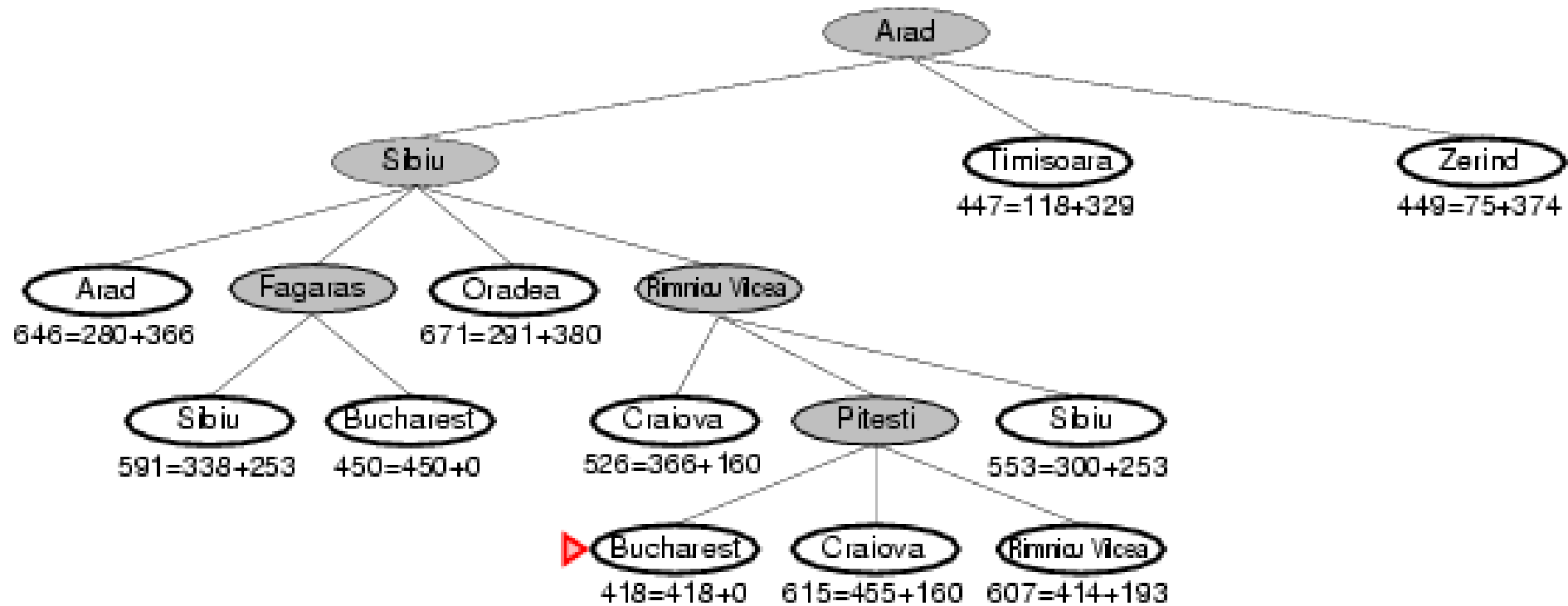
A* search example



A* search example

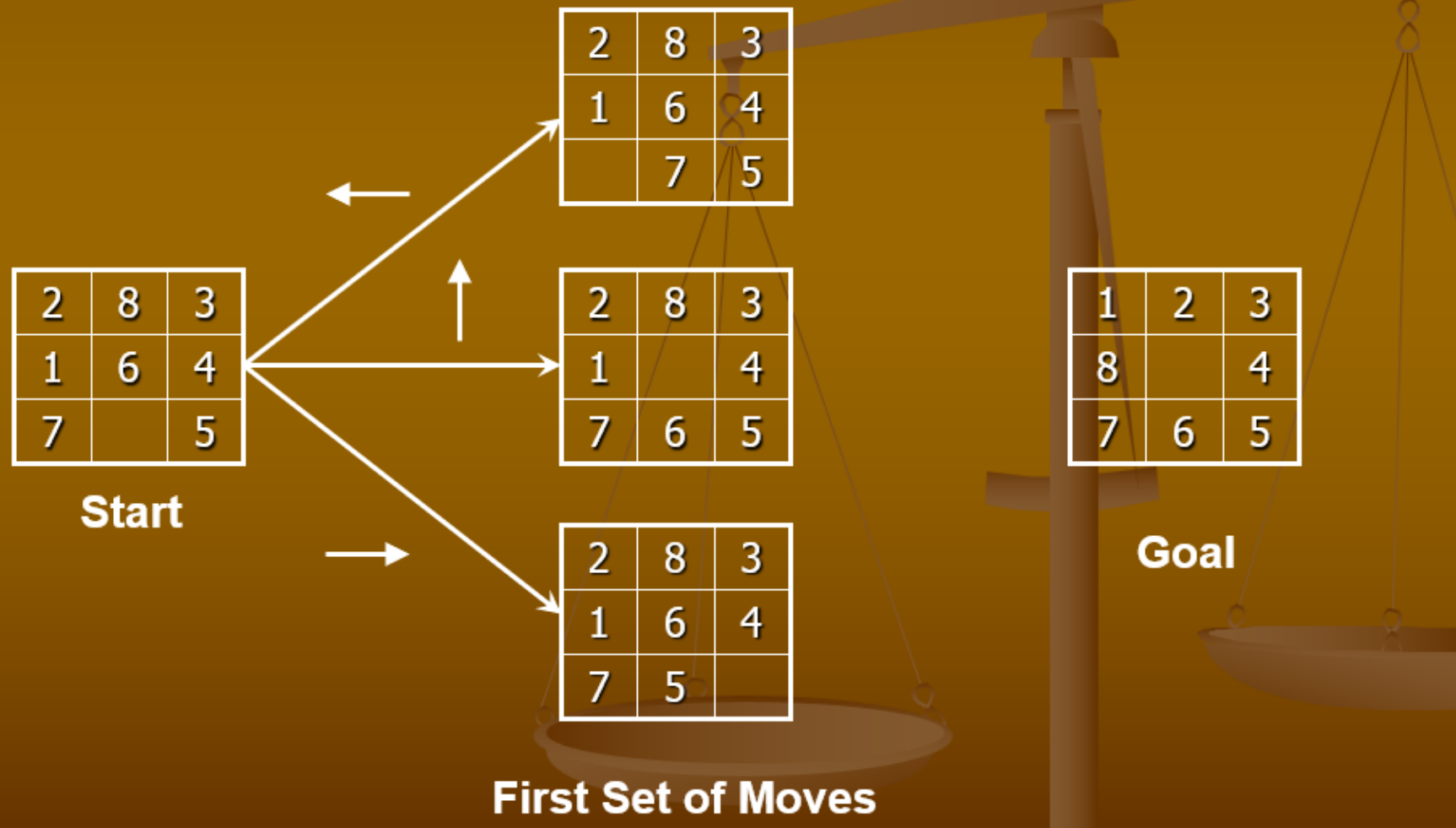


A* search example



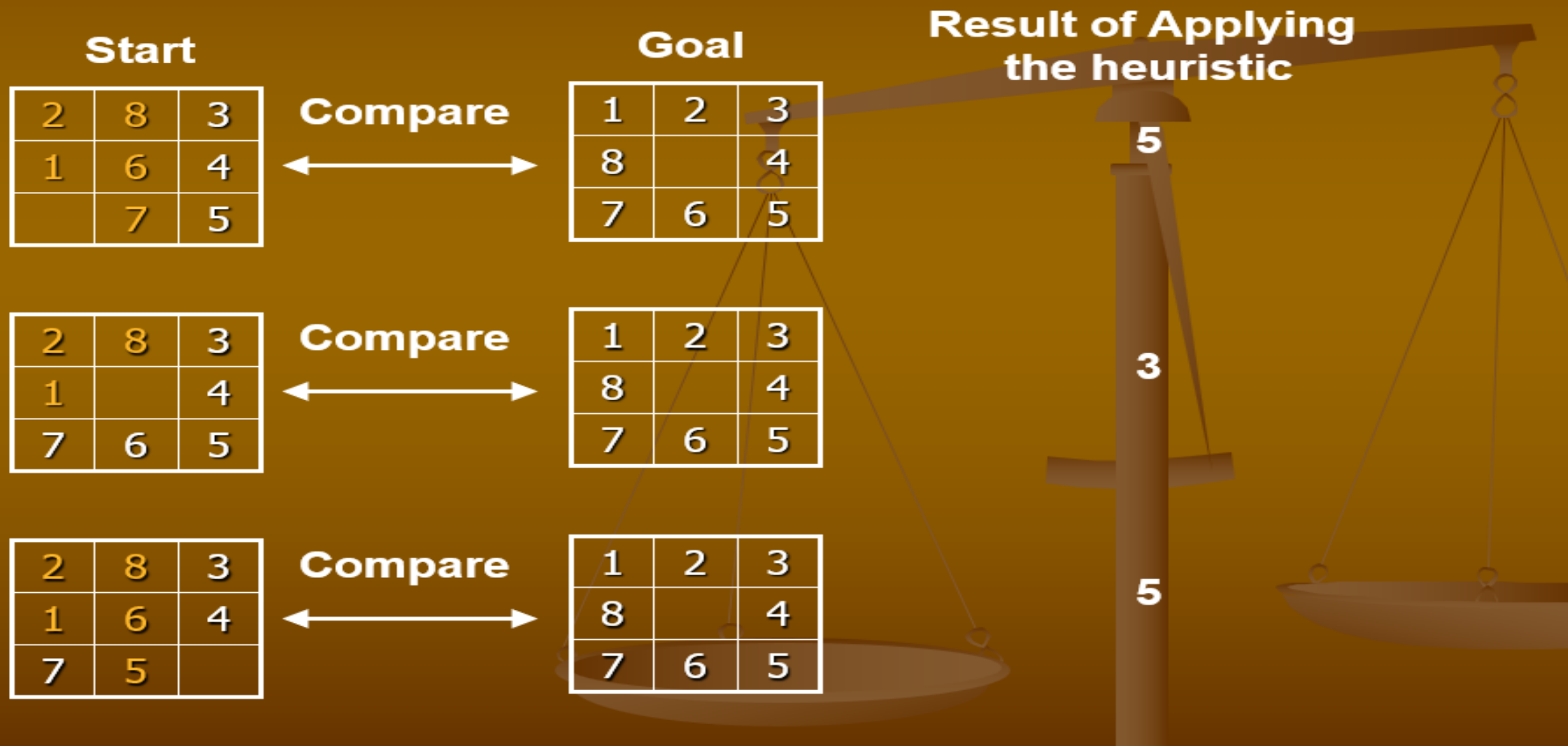
8-Puzzle with Heuristic Search

- Lets look at the performance of several different heuristics for solving the 8-puzzle.
- The figure 4.6 shows a start and goal state for the 8-puzzle, along with the first three states generated in the search.



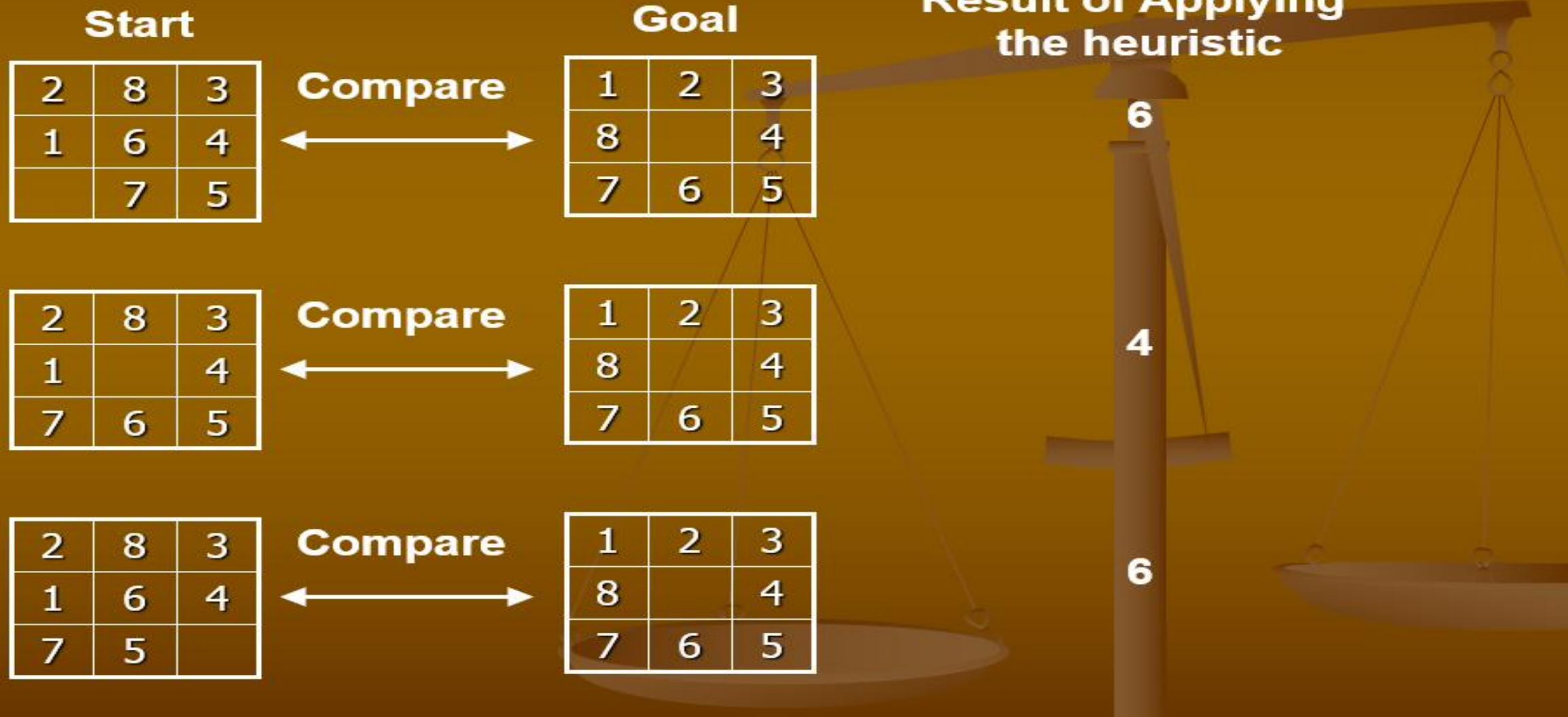
8-Puzzle with Heuristic Search

- Heuristic #1: Count the tiles out of place in each state when it is compared with the goal.



8-Puzzle with Heuristic Search

- Heuristic #2: Sum all the distances by which the tiles are out of place (one for each square, a tile be moved to reach its position in the goal state).



8-Puzzle with Heuristic Search

- Heuristic #3: Both above heuristics can be criticized for failing to acknowledge the difficulty of tile reversals.
- Tile reversals means if two tiles are next to each other and the goal requires their being in opposite locations.
- It takes (many) more than two moves to put them back in place, as the tile must “go around” each other
- The following figure shows an 8-puzzle with a goal and two tile reversals, i.e. 1 and 2; 5 and 6.



Level of search
 $g(n) =$

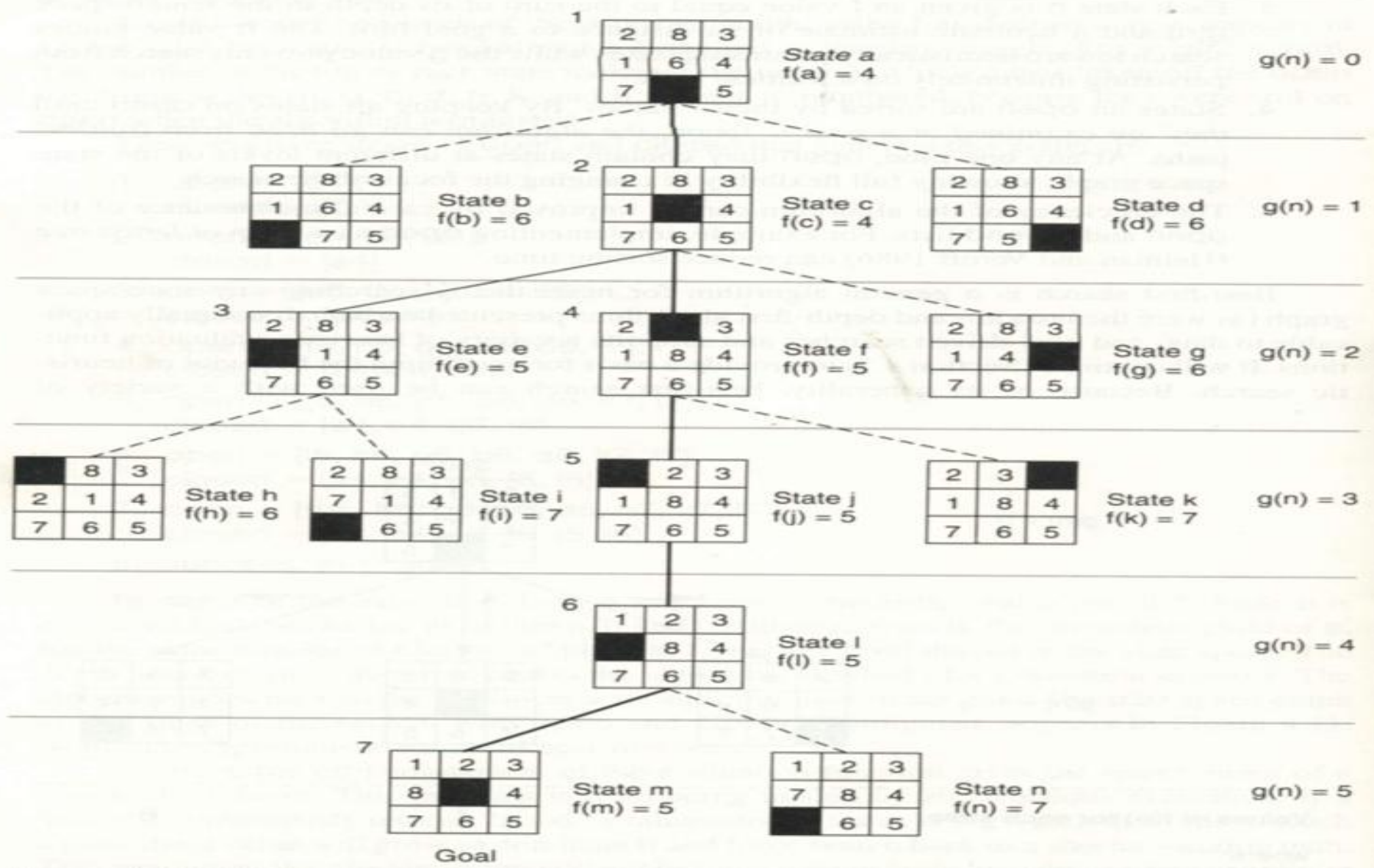


Figure 4.10 State space generated in heuristic search of the 8-puzzle graph.

Algorithm A for 8-Puzzle



Conditions for Optimality

- Admissible
- Consistency



A^* is Admissible

- A^* will never overestimate the cost of reaching the goal.
- The cost to reach goal is guaranteed to be lower or equal to the actual cost.
- This property ensures that **the algorithm will eventually find the optimal solution, if one exists.**



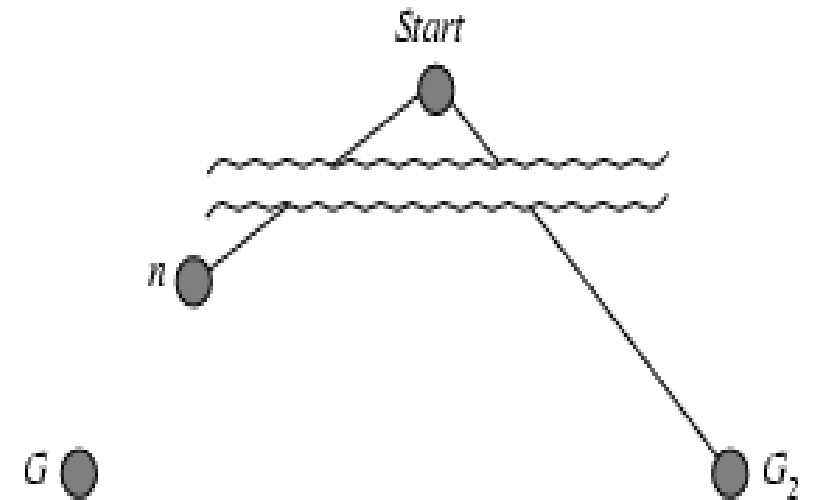
Admissible heuristics

- A heuristic $h(n)$ is **admissible** if for every node n , $h(n) \leq h^*(n)$, where $h^*(n)$ is the **true/actual** cost to reach the goal state from n and $h(n)$ is the *estimated cost to reach the goal*
- An admissible heuristic **never overestimates** the cost to reach the goal, i.e., it is **optimistic**
- Example: $h_{SLD}(n)$ (never overestimates the actual road distance)
- **Theorem:** If $h(n)$ is admissible, A^* using TREE-SEARCH is optimal

Optimality of A^* (proof)

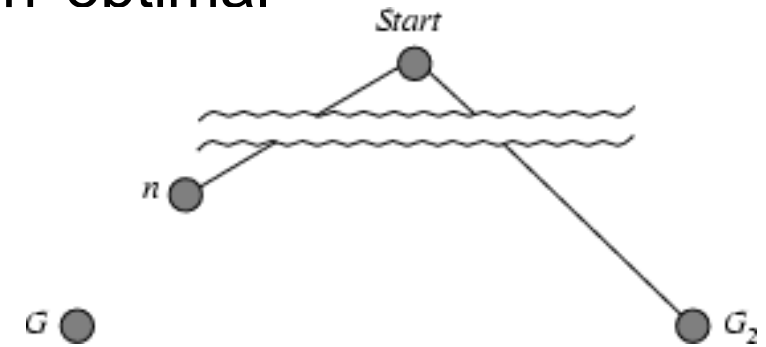
- Suppose some suboptimal goal G_2 has been generated and is in the fringe. Let n be an unexpanded node in the fringe such that n is on a shortest path to an optimal goal G .

- $f(G_2) = g(G_2)$ since $h(G_2) = 0$
- $g(G_2) > g(G)$ since G_2 is suboptimal
- $f(G) = g(G)$ since $h(G) = 0$
- $f(G_2) > f(G)$ from above



Optimality of A^* (proof)

- Suppose some suboptimal goal G_2 has been generated and is in the fringe. Let n be an unexpanded node in the fringe such that n is on a shortest path to an optimal goal G .



- $f(G_2) > f(G)$ from above
- $h(n) \leq h^*(n)$ since h is admissible, h^* is minimal distance/actual cost
- $g(n) + h(n) \leq g(n) + h^*(n)$
- $f(n) \leq f(G)$

Hence $f(G_2) > f(n)$, and A^* will never select G_2 for expansion

Consistent heuristics

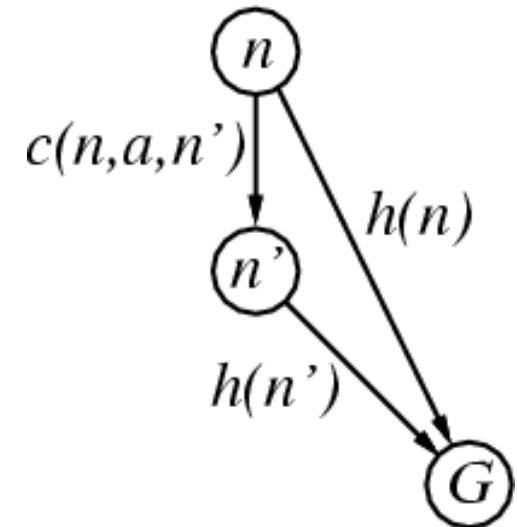
- A heuristic is **consistent** if for every node n , every successor n' of n generated by any action a , the estimated cost of reaching the goal from node n , $h(n)$, is not greater than the step cost of getting to n' , $(g(n')) + \text{estimated cost of reaching the goal from } n' (f(n'))$

$$h(n) \leq g(n') + h(n')$$

$$h(n) \leq c(n, a, n') + h(n')$$

- Intuition: can't do worse than going through n' .
- If h is consistent, we have

$$\begin{aligned} f(n') &= g(n') + h(n') = g(n) + c(n, a, n') + h(n') \\ &\geq g(n) + h(n) = f(n) \end{aligned}$$

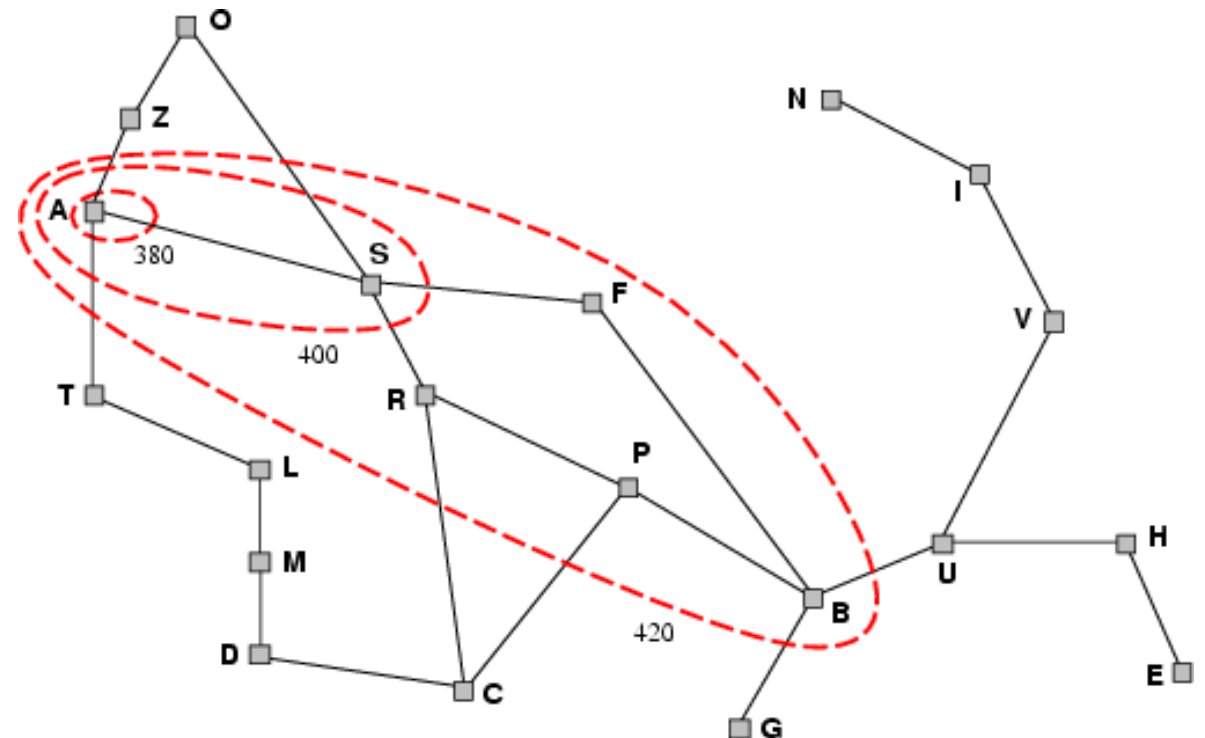


triangle inequality

Theorem: If $h(n)$ is consistent, A* using GRAPH-SEARCH is optimal

Optimality of A^*

- A^* expands nodes in order of increasing f value
 - This means that nodes with lower f -values are expanded before nodes with higher f -values.
- Gradually adds "f-contours" of nodes
- Contour i has all nodes with $f=f_i$, where $f_i < f_{i+1}$



Monotonicity

- A heuristic function h is monotone if
 1. $h(n_i) - h(n_j) \leq \text{cost}(n_i, n_j)$
where $\text{cost}(n_i, n_j)$ is the actual cost (in number of moves) of going from state n_i to n_j .
 2. The heuristic evaluation of the goal state is zero i.e. $h(\text{goal}) = 0$.
- A simple argument can show us that any monotonic heuristic is admissible.
- This argument considers any path in the space as a sequence of states $s_1, s_2, \dots, s_g$, where s_1 is the start state and s_g is the goal state. Now consider:

s_1 to s_2	$h(s_1) - h(s_2) \leq \text{cost}(s_1, s_2)$ by monotonic property
----------------	--

s_2 to s_3	$h(s_2) - h(s_3) \leq \text{cost}(s_2, s_3)$ by monotonic property
----------------	--

.....

$s_{(g-1)}$ to s_g	$h(s_{(g-1)}) - h(s_g) \leq \text{cost}(s_{(g-1)}, s_g)$ by monotonic property
----------------------	--

Adding up each column (knowing that $h(s_g) = 0$), we get

s_1 to s_g	$h(s_1) \leq \text{cost}(s_1, s_g)$
----------------	-------------------------------------

- This means that monotonic heuristic h is A^* and admissible.



Properties of A*

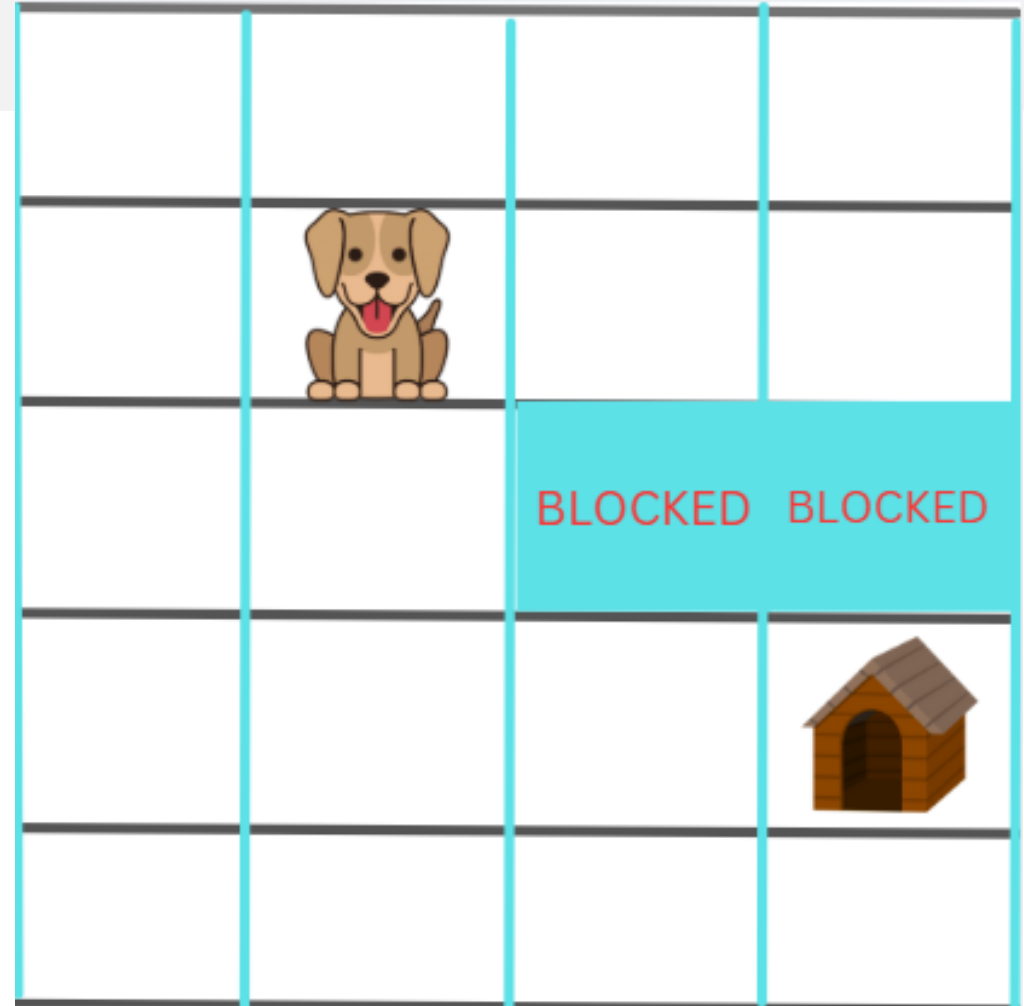
- Complete? Yes (unless there are infinitely many nodes with $f \leq f(G)$)
- Time? Exponential
- Space? Keeps all nodes in memory
- Optimal? Yes



Find Home for Kuki using A*?

$G(n) = ?$

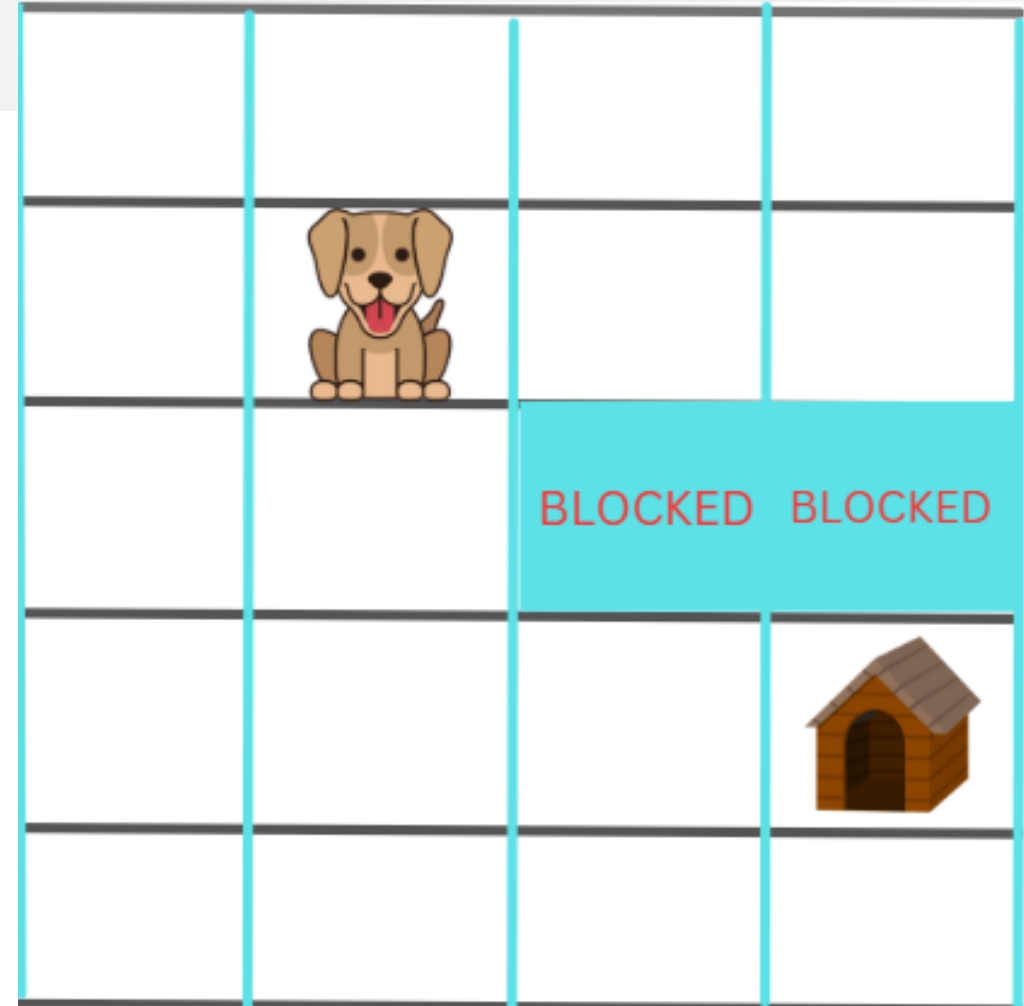
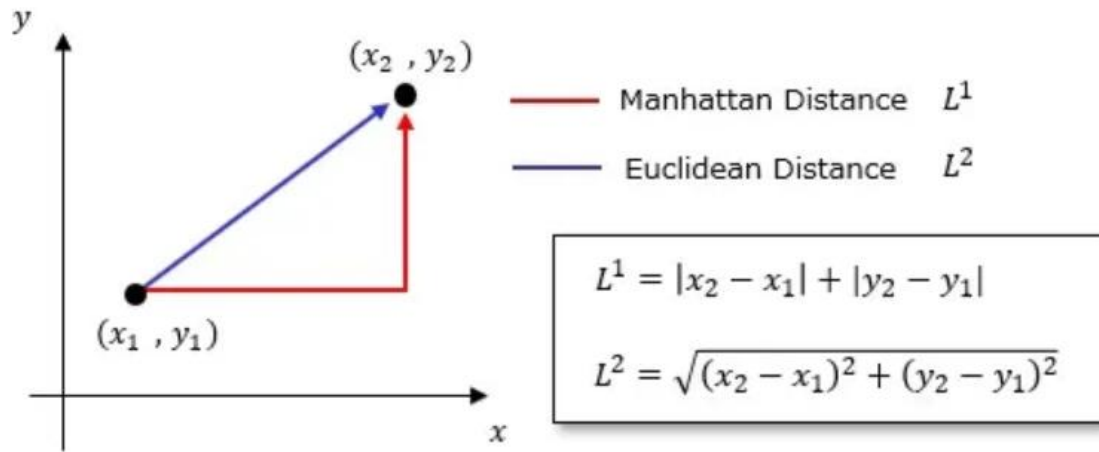
$H(n) = ?$



Find Home for Kuki using A*?

$G(n)$ = No of Step Taken?

$H(n)$ =



Choose one of the distance functions as heuristic value and prove that heuristic is admissible?



Relaxed problems

- A problem with fewer restrictions on the actions is called a **relaxed problem**
- The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem
- If the rules of the 8-puzzle are relaxed so that a tile can move **anywhere**, then $h_1(n)$ gives the shortest solution
- If the rules are relaxed so that a tile can move to **any adjacent square**, then $h_2(n)$ gives the shortest solution